

boba_extension_dim02

Dimension 02: qBitTorrent WebUI API v2 Complete Reference

Research Date: 2025-07-28 **Scope:** Browser extension integration for sending/managing torrents via qBittorrent WebUI API **Target Versions:** qBittorrent v4.1+ through v5.2+ **Primary Documentation:** [https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-\(qBittorrent-4.1\)](https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-(qBittorrent-4.1))

Table of Contents

1. [General Information](#)
 2. [Authentication Flow](#)
 3. [Adding Torrents](#)
 4. [Adding Magnet Links](#)
 5. [Uploading .torrent Files](#)
 6. [Monitoring Torrents](#)
 7. [Torrent Management](#)
 8. [Application Preferences](#)
 9. [Category Management](#)
 10. [Tag Management](#)
 11. [Search API](#)
 12. [RSS API](#)
 13. [Log API](#)
 14. [Error Handling](#)
 15. [Version Differences \(4.x vs 5.x\)](#)
 16. [Python qbittorrent-api Library](#)
 17. [WebSocket/SSE and Real-Time Updates](#)
 18. [Complete Working Examples](#)
 19. [Browser Extension Integration Notes](#)
 20. [References](#)
-

1. General Information

Base URL Format

All API methods follow the format `/api/v2/APIName/methodName`.

`http(s)://<host>:<port>/api/v2/<namespace>/<method>`

Example: `http://localhost:8080/api/v2/torrents/info`

HTTP Methods

- **GET:** For read operations (retrieving data)
- **POST:** For state-mutating operations, file uploads, or when request data is too large for GET

Critical: Starting with qBittorrent v4.4.4, the server returns **405 Method Not Allowed** when the wrong request method is used. [6]

Authentication Requirement

All API methods require authentication **except** `/api/v2/auth/login`. [6]

API Namespaces

Namespace	Description
auth	Authentication (login/logout)
app	Application info, preferences, build info
torrents	Torrent management (CRUD, control)
sync	Incremental data synchronization
transfer	Global transfer info, speed limits
log	Log entries (main log, peer log)
rss	RSS feeds and auto-downloading rules
search	Search plugins and search jobs

2. Authentication Flow

qBittorrent uses **cookie-based authentication** by default. Starting from v5.2.0, **API key authentication** is also supported.

2.1 Cookie-Based Authentication (All Versions)

Login

Endpoint: POST /api/v2/auth/login

Parameter	Type	Description
username	string	WebUI username
password	string	WebUI password

Returns:

HTTP Status	Scenario
403	User's IP is banned for too many failed login attempts
200	Success - SID cookie is set

curl Example:

```
# Step 1: Login and capture the SID cookie
curl -i --header 'Referer: http://localhost:8080' \
  --data 'username=admin&password=adminadmin' \
  http://localhost:8080/api/v2/auth/login

# Response:
# HTTP/1.1 200 OK
# Set-Cookie: SID=hBc7TxF76ERhvIw0jQQ4LZ7Z1jQUV0tQ; path=/
# Content-Length: 3
# Content-Type: text/plain; charset=UTF-8
#
# Ok.

# Step 2: Use the cookie for subsequent requests
curl http://localhost:8080/api/v2/torrents/info \
  --cookie "SID=hBc7TxF76ERhvIw0jQQ4LZ7Z1jQUV0tQ"
```

JavaScript Fetch Example:

```
// Browser extension: login to qBittorrent
async function qbittorrentLogin(baseUrl, username, password) {
  const loginUrl = `${baseUrl}/api/v2/auth/login`;

  const formData = new URLSearchParams();
  formData.append('username', username);
  formData.append('password', password);

  const response = await fetch(loginUrl, {
    method: 'POST',
    headers: {
```

```

        'Referer': baseUrl, // CRITICAL: Referer/Origin must match
                             Host
        'Content-Type': 'application/x-www-form-urlencoded',
    },
    body: formData.toString(),
    credentials: 'include', // Important: accept and send cookies
});

if (response.status === 403) {
    throw new Error('IP banned - too many failed login attempts');
}

return response.ok; // true if 200
}

```

CRITICAL CSRF NOTE: The Referer or Origin header **must** be set to the exact same domain and port as the Host header. If using a reverse proxy, the Origin/Referer must match what qBittorrent sees as the Host. A null Origin is also accepted by qBittorrent for edge cases. ^[6] ^[47]

Logout

Endpoint: POST /api/v2/auth/logout

Parameter	Type	Description
None	-	-

Returns: 200 OK in all scenarios

```
curl -X POST http://localhost:8080/api/v2/auth/logout \
  --cookie "SID=<your_sid>"
```

Cookie Handling for Browser Extensions

```

// For browser extensions, use the Fetch API with credentials
class QBittorrentClient {
    constructor(baseUrl) {
        this.baseUrl = baseUrl;
    }

    async login(username, password) {
        const response = await fetch(`${this.baseUrl}/api/v2/auth/login`, {
            method: 'POST',
            headers: {
                'Referer': this.baseUrl,
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: new URLSearchParams({ username, password }).toString(),
        });
    }
}

```

```

        credentials: 'include',
    });
    return response.ok;
}

async apiCall(endpoint, method = 'GET', body = null,
    contentType = null) {
    const options = {
        method,
        credentials: 'include', // Always send cookies
        headers: {
            'Referer': this.baseUrl,
        },
    };

    if (body && contentType) {
        options.headers['Content-Type'] = contentType;
        options.body = body;
    } else if (body && method === 'POST') {
        options.headers['Content-Type'] = 'application/x-www-form-urlencoded';
        options.body = new URLSearchParams(body).toString();
    }

    const response = await fetch(`${this.baseUrl}/api/v2${
        {endpoint}}`, options);

    if (response.status === 403) {
        // Session expired or not authenticated
        throw new Error('Not authenticated - login required');
    }

    return response;
}
}

```

2.2 API Key Authentication (qBittorrent v5.2.0+, API v2.14.1+)

Starting from qBittorrent v5.2.0, a stateless API key authentication method is available. ^[119]

API Key Format

- 32 characters long
- Prefix: qbt_ + 28 random alphanumeric characters
- Generated with 160 bits of entropy
- Only **one** API key is supported at a time
- Rotating the key immediately invalidates the previous key

Usage

Specify the key in the Authorization header using the Bearer scheme:

```
Authorization: Bearer qbt_abc123...xyz789
```

Generating an API Key

1. Open qBittorrent Preferences
2. Click on WebUI
3. Navigate to the API Key section
4. Click the Generate icon

curl Example

```
curl http://localhost:8080/api/v2/torrents/info \
-H "Authorization: Bearer qbt_abc123...xyz789"
```

JavaScript Fetch Example

```
class QBittorrentAPIKeyClient {
  constructor(baseUrl, apiKey) {
    this.baseUrl = baseUrl;
    this.apiKey = apiKey;
  }

  async apiCall(endpoint, method = 'GET', body = null) {
    const options = {
      method,
      headers: {
        'Authorization': `Bearer ${this.apiKey}`,
      },
    };

    if (body) {
      options.body = body;
    }

    return fetch(`${this.baseUrl}/api/v2${endpoint}`, options);
  }

  async addTorrentUrl(urls, options = {}) {
    const formData = new FormData();
    formData.append('urls', urls);
    if (options.savepath) formData.append('savepath', options.savepath);
    if (options.category) formData.append('category', options.category);
    if (options.paused) formData.append('paused', 'true');
  }
}
```

```

    return this.apiCall('/torrents/add', 'POST', formData);
  }
}

```

API Key Limitations

- Cannot be used to fetch the WebUI or other static assets
- Cannot interact with auth endpoints (login, logout)
- Designed for API-only clients ^[119]

3. Adding Torrents

3.1 Primary Endpoint: /api/v2/torrents/add

Method: POST **Content-Type:** multipart/form-data (required for file uploads; also works for URLs)

Parameters

Parameter	Type	Required	Description
urls	string	* (if no torrents)	URLs separated by newlines (\n). Supports HTTP/HTTPS URLs to .torrent files and magnet: URIs
torrents	binary	* (if no urls)	Raw data of .torrent file. Can be provided multiple times for batch upload
savepath	string	optional	Download folder path (absolute)
cookie	string	optional	Cookie sent to download the .torrent file from URL
category	string	optional	Category for the torrent
tags	string	optional	Tags for the torrent, comma-separated (e.g., tag1, tag2, tag3)
skip_checking	string	optional	Skip hash checking. Values:

Parameter	Type	Required	Description
			true, false (default)
paused	string	optional	Add torrent in paused state. Values: true, false (default)
root_folder	string	optional	Create root folder. Values: true, false, unset (default)
rename	string	optional	Rename the torrent
upLimit	integer	optional	Upload speed limit in bytes/second
dlLimit	integer	optional	Download speed limit in bytes/ second
ratioLimit	float	optional	Share ratio limit (since API v2.8.1)
seedingTimeLimit	integer	optional	Seeding time limit in minutes (since API v2.8.1)
autoTMM	bool	optional	Enable Automatic Torrent Management
sequentialDownload	string	optional	Enable sequential download. Values: true, false (default)
firstLastPiecePrio	string	optional	Prioritize first/last piece. Values: true, false (default)
content_layout	string	optional	Content layout: Original, Subfolder, NoSubfolder (supersedes root_folder since API v2.7)
downloadPath	string	optional	Download folder for incomplete torrents (since v5.0)
stopCondition	string	optional	Stop condition: None, MetadataReceived,

Parameter	Type	Required	Description
			FilesChecked (since v5.0)
ssl_certificate	string	optional	Peer certificate in PEM format (since v5.0, API v2.10.4)
ssl_private_key	string	optional	Peer private key (since v5.0, API v2.10.4)
ssl_dh_params	string	optional	Diffie-Hellman parameters (since v5.0, API v2.10.4)
is_stopped	bool	optional	Add torrent in stopped state (since API v2.11.0, v5.0+)
forced	bool	optional	Add torrent in forced state (since API v2.11.0)

Note on root_folder vs content_layout: content_layout supersedes root_folder since API v2.7. Values are Original (use original layout), Subfolder (create subfolder), NoSubfolder (no subfolder). ¹³

Returns

HTTP Status	Scenario
415	Torrent file is not valid
200	Success (all other scenarios)

Important: The response body on success is typically 0k. or empty. A 200 status does not guarantee the torrent was successfully added (e.g., URL may be unreachable). ^[6]

4. Adding Magnet Links

Magnet links are passed through the `urls` parameter of `/api/v2/torrents/add`. Multiple URLs/magnets can be provided, separated by newline characters (`\n`).

Single Magnet Link

```
curl -X POST http://localhost:8080/api/v2/torrents/add \
  --cookie "SID=<your_sid>" \
  --data 'urls=magnet:?
        xt=urn:btih:08ada5a7a6183aae1e09d831df6748d566095a10&dn=Sintel'
```

Multiple Magnet Links

```
curl -X POST http://localhost:8080/api/v2/torrents/add \
  --cookie "SID=<your_sid>" \
  --data '$'urls=magnet:?xt=urn:btih:hash1&dn=Name1\nmagnet:?
        xt=urn:btih:hash2&dn=Name2'
```

With Options (Category, Save Path, Paused)

```
curl -X POST http://localhost:8080/api/v2/torrents/add \
  --cookie "SID=<your_sid>" \
  --data 'urls=magnet:?xt=urn:btih:...&dn=Movie' \
  --data 'category=Movies' \
  --data 'savepath=/downloads/Movies' \
  --data 'paused=true' \
  --data 'autoTMM=true'
```

JavaScript Fetch - Add Magnet Link

```
async function addMagnetLink(client, magnetUri, options = {}) {
  const formData = new FormData();
  formData.append('urls', magnetUri);

  // Optional parameters
  if (options.savepath) formData.append('savepath',
    options.savepath);
  if (options.category) formData.append('category',
    options.category);
  if (options.tags) formData.append('tags', options.tags);
  if (options.paused) formData.append('paused', 'true');
  if (options.skipChecking) formData.append('skip_checking',
    'true');
  if (options.autoTMM) formData.append('autoTMM', 'true');
  if (options.sequentialDownload)
    formData.append('sequentialDownload', 'true');
  if (options.upLimit) formData.append('upLimit',
    options.upLimit.toString());
  if (options.dlLimit) formData.append('dlLimit',
    options.dlLimit.toString());
  if (options.rename) formData.append('rename', options.rename);
  if (options.ratioLimit) formData.append('ratioLimit',
    options.ratioLimit.toString());
  if (options.seedingTimeLimit)
    formData.append('seedingTimeLimit',
    options.seedingTimeLimit.toString());
}
```

```

const response = await fetch(`${client.baseUrl}/api/v2/torrents/
  add`, {
  method: 'POST',
  credentials: 'include',
  headers: {
    'Referer': client.baseUrl,
  },
  body: formData,
});

if (response.status === 415) {
  throw new Error('Invalid torrent data');
}

return response.ok; // true if 200
}

```

Browser Extension: Context Menu Handler for Magnet Links

```

// manifest.json - context menu for magnet links
chrome.runtime.onMessage.addListener((message, sender,
  sendResponse) => {
  if (message.type === 'ADD_MAGNET') {
    addMagnetToQBittorrent(message.magnetUri, message.options)
      .then(result => sendResponse({ success: true, result }))
      .catch(error => sendResponse({ success: false, error:
        error.message }));
    return true; // async response
  }
});

async function addMagnetToQBittorrent(magnetUri, options = {}) {
  const settings = await chrome.storage.sync.get([
    'qbBaseUrl', 'qbUsername', 'qbPassword', 'defaultCategory',
    'defaultSavePath'
  ]);

  // Login
  const loginForm = new URLSearchParams();
  loginForm.append('username', settings.qbUsername);
  loginForm.append('password', settings.qbPassword);

  const loginResponse = await fetch(`${settings.qbBaseUrl}/api/v2/
    auth/login`, {
    method: 'POST',
    headers: { 'Referer': settings.qbBaseUrl },
    body: loginForm.toString(),
    credentials: 'include',
  });
}

```

```

    if (!loginResponse.ok) {
        throw new Error('Login failed');
    }

    // Add magnet
    const formData = new FormData();
    formData.append('urls', magnetUri);
    if (settings.defaultCategory) formData.append('category',
        settings.defaultCategory);
    if (settings.defaultSavePath) formData.append('savepath',
        settings.defaultSavePath);

    // Merge any per-torrent options
    Object.entries(options).forEach(([key, value]) => {
        if (value !== undefined && value !== null) {
            formData.append(key, String(value));
        }
    });

    const response = await fetch(`${settings.qbBaseUrl}/api/v2/
        torrents/add`, {
        method: 'POST',
        credentials: 'include',
        headers: { 'Referer': settings.qbBaseUrl },
        body: formData,
    });

    if (response.status === 415) {
        throw new Error('Invalid magnet URI');
    }
    if (!response.ok) {
        throw new Error(`Failed to add torrent: ${response.status}`);
    }

    return { added: true };
}

```

5. Uploading .torrent Files

5.1 Multipart Form Data Upload

The torrents parameter accepts raw binary data of a .torrent file. Multiple files can be uploaded in a single request by including multiple torrents fields.

curl Example - Single File

```

curl -X POST http://localhost:8080/api/v2/torrents/add \
    --cookie "SID=<your_sid>" \

```

```
-F "torrents=@/path/to/file.torrent" \  
-F "category=Movies" \  
-F "savepath=/downloads/Movies" \  
-F "paused=false"
```

curl Example - Multiple Files

```
curl -X POST http://localhost:8080/api/v2/torrents/add \  
  --cookie "SID=<your_sid>" \  
  -F "torrents=@/path/to/file1.torrent" \  
  -F "torrents=@/path/to/file2.torrent" \  
  -F "category=TV" \  
  -F "autoTMM=true"
```

5.2 Binary Data Upload (without file system access)

For browser extensions that have the .torrent file as a Blob or ArrayBuffer (e.g., downloaded from a website):

```
async function uploadTorrentFile(baseUrl, fileBlob, filename,  
  options = {}) {  
  const formData = new FormData();  
  formData.append('torrents', fileBlob, filename);  
  
  if (options.category) formData.append('category',  
    options.category);  
  if (options.savepath) formData.append('savepath',  
    options.savepath);  
  if (options.paused) formData.append('paused', 'true');  
  if (options.skipChecking) formData.append('skip_checking',  
    'true');  
  if (options.autoTMM) formData.append('autoTMM', 'true');  
  if (options.tags) formData.append('tags', options.tags);  
  
  const response = await fetch(`${baseUrl}/api/v2/torrents/add`, {  
    method: 'POST',  
    credentials: 'include',  
    headers: {  
      'Referer': baseUrl,  
    },  
    body: formData,  
  });  
  
  return response.ok;  
}
```

```
// Usage: Upload from a downloaded file in a browser extension  
async function handleTorrentFileDownload(arrayBuffer, filename) {  
  const settings = await chrome.storage.sync.get(['qbBaseUrl']);  
  const blob = new Blob([arrayBuffer], { type: 'application/x-  
    bittorrent' });  
  return uploadTorrentFile(settings.qbBaseUrl, blob, filename, {
```

```

        category: 'Downloaded',
    });
}

```

5.3 Raw Bytes via Python

```
import requests
```

```
# Session to persist cookies
session = requests.Session()
```

```
# Login
session.post(
    'http://localhost:8080/api/v2/auth/login',
    data={'username': 'admin', 'password': 'adminadmin'},
    headers={'Referer': 'http://localhost:8080'}
)
```

```
# Upload torrent file from disk
with open('/path/to/file.torrent', 'rb') as f:
    response = session.post(
        'http://localhost:8080/api/v2/torrents/add',
        files={'torrents': ('file.torrent', f, 'application/x-
bittorrent')},
        data={'category': 'Movies', 'savepath': '/downloads/
Movies'}
    )
```

```
print(f"Status: {response.status_code}") # 200 = added
```

5.4 Multipart Boundary Notes

The Content-Type: multipart/form-data boundary in the POST body is preceded by two hyphens, and the end of the body is closed by two hyphens appended to the boundary string.

Content-Type: multipart/form-data; boundary=--AqhE2AFEJbRxE4xx

----AqhE2AFEJbRxE4xx

Content-Disposition: form-data; name="urls"

https://example.com/file.torrent

----AqhE2AFEJbRxE4xx

Content-Disposition: form-data; name="category"

Movies

----AqhE2AFEJbRxE4xx--

Note: When using FormData in JavaScript or files parameter in Python requests, the boundary is handled automatically.
[⁶]

6. Monitoring Torrents

6.1 Get Torrent List: /api/v2/torrents/info

Method: GET

Query Parameters

Parameter	Type	Required	Description
filter	string	optional	Filter by state: all, downloading, seeding, completed, paused, active, inactive, resumed, stalled, stalled_uploading, stalled_downloading, errored
category	string	optional	Filter by category (empty string = "without category"; no param = "any category")
tag	string	optional	Filter by tag (since API v2.8.3). Empty string = "without tag"; no param = "any tag"
sort	string	optional	Sort by any response JSON field
reverse	bool	optional	Enable reverse sorting (default: false)
limit	integer	optional	Limit number of torrents returned
offset	integer	optional	Set offset (if < 0, offset from end)
hashes	string	optional	

Parameter	Type	Required	Description
			Filter by specific hash(es), separated by \

Filter value differences: In qBittorrent v5.0+, paused filter was replaced with stopped, and a new running filter was added. [¹²¹]

Response Fields

The response is a JSON array of torrent objects. Each object has:

Property	Type	Description
added_on	integer	Unix timestamp when torrent was added
amount_left	integer	Data left to download (bytes)
auto_tmm	bool	Whether Automatic Torrent Management is enabled
availability	float	Percentage of file pieces currently available (0.0 to 1.0)
category	string	Category name
completed	integer	Amount of transfer data completed (bytes)
completion_on	integer	Unix timestamp when torrent completed
content_path	string	Absolute path of torrent content
dl_limit	integer	Download speed limit (bytes/s), -1 = unlimited
dlspeed	integer	Current download speed (bytes/s)
downloaded	integer	Total data downloaded (bytes)
downloaded_session	integer	Data downloaded this session (bytes)
eta	integer	Estimated time to completion (seconds)
f_l_piece_prio	bool	Whether first/last piece priority is enabled
force_start	bool	Whether force start is enabled
hash	string	Torrent hash (infohash v1)

Property	Type	Description
isPrivate	bool	Whether torrent is from a private tracker (v5.0+)
infohash_v1	string	Infohash v1 (v5.0+)
infohash_v2	string	Infohash v2 (v5.0+)
last_activity	integer	Unix timestamp of last upload/download activity
magnet_uri	string	Magnet URI for this torrent
max_ratio	float	Maximum share ratio for auto-stopping
max_seeding_time	integer	Maximum seeding time (seconds)
name	string	Torrent name
num_complete	integer	Number of seeds in the swarm
num_incomplete	integer	Number of leechers in the swarm
num_leechs	integer	Number of connected leechers
num_seeds	integer	Number of connected seeds
priority	integer	Torrent queue priority (-1 if queueing disabled)
progress	float	Download progress (0.0 to 1.0)
ratio	float	Share ratio (max: 9999)
ratio_limit	float	Per-torrent ratio limit
reannounce	integer	Seconds until next tracker reannounce (v5.0+)
save_path	string	Path where torrent data is stored
seeding_time	integer	Time spent seeding (seconds)
seeding_time_limit	integer	Seeding time limit for this torrent
seen_complete	integer	Unix timestamp when torrent was last seen complete
seq_dl	bool	Whether sequential download is enabled
size	integer	Total size of selected files (bytes)
state	string	Current torrent state (see state table below)

Property	Type	Description
super_seeding	bool	Whether super seeding is enabled
tags	string	Comma-separated list of tags
time_active	integer	Total active time (seconds)
total_size	integer	Total size of all files including unselected (bytes)
tracker	string	First working tracker URL (empty if none)
trackers_count	integer	Number of trackers (v5.0+)
up_limit	integer	Upload speed limit (bytes/s), -1 = unlimited
uploaded	integer	Total data uploaded (bytes)
uploaded_session	integer	Data uploaded this session (bytes)
upspeed	integer	Current upload speed (bytes/s)

[Source: GitHub Wiki - WebUI API (qBittorrent 4.1), “Get torrent list” section] ^[6] ^[164]

Torrent States

State	Description
error	Some error occurred (applies to paused torrents)
missingFiles	Torrent data files are missing
uploading	Torrent is being seeded with active data transfer
pausedUP	Torrent is paused and has finished downloading
stoppedUP	Same as pausedUP but renamed in v5.0+ (API v2.11.0)
queuedUP	Torrent is queued for upload
stalledUP	Torrent is seeding but no connections
checkingUP	Torrent finished downloading and is being checked
forcedUP	Force upload, ignores queue limit
allocating	Allocating disk space for download
downloading	Torrent is being downloaded with active data transfer

State	Description
metaDL	Downloading metadata (magnet links)
pausedDL	Torrent is paused and has NOT finished downloading
stoppedDL	Same as pausedDL but renamed in v5.0+ (API v2.11.0)
queuedDL	Torrent is queued for download
stalledDL	Torrent is downloading but no connections
checkingDL	Torrent is being checked (has NOT finished downloading)
forcedDL	Force download, ignores queue limit
checkingResumeData	Checking resume data on qBittorrent startup
moving	Torrent is being moved to another location
unknown	Unknown status

v5.0 Breaking Change: pausedUP was renamed to stoppedUP and pausedDL was renamed to stoppedDL. Both old and new names may appear depending on version. ^[137] ^[121]

curl Examples

Get all torrents

```
curl http://localhost:8080/api/v2/torrents/info \
  --cookie "SID=<your_sid>"
```

Filter by downloading state

```
curl "http://localhost:8080/api/v2/torrents/info?
  filter=downloading" \
  --cookie "SID=<your_sid>"
```

Filter by category, sorted by ratio descending

```
curl "http://localhost:8080/api/v2/torrents/info?
  category=Movies&sort=ratio&reverse=true" \
  --cookie "SID=<your_sid>"
```

Get specific torrent by hash

```
curl "http://localhost:8080/api/v2/torrents/info?
  hashes=abc123..." \
  --cookie "SID=<your_sid>"
```

Paginated query

```
curl "http://localhost:8080/api/v2/torrents/info?
  limit=50&offset=0&sort=added_on&reverse=true" \
  --cookie "SID=<your_sid>"
```

JavaScript Example - Poll Torrent List

```
async function getTorrentList(baseUrl, options = {}) {
  const params = new URLSearchParams();
  if (options.filter) params.append('filter', options.filter);
  if (options.category) params.append('category',
    options.category);
  if (options.tag) params.append('tag', options.tag);
  if (options.sort) params.append('sort', options.sort);
  if (options.reverse) params.append('reverse', 'true');
  if (options.limit) params.append('limit',
    options.limit.toString());
  if (options.offset !== undefined) params.append('offset',
    options.offset.toString());
  if (options.hashes) params.append('hashes', options.hashes);

  const queryString = params.toString();
  const url = `${baseUrl}/api/v2/torrents/info${queryString ? '?' : ''}
    + queryString : ''`;

  const response = await fetch(url, {
    method: 'GET',
    credentials: 'include',
    headers: { 'Referer': baseUrl },
  });

  if (!response.ok) {
    throw new Error(`Failed to fetch torrents: ${response.status}
    `);
  }

  return response.json(); // Returns array of torrent objects
}

// Usage: Get active downloads
const downloading = await getTorrentList('http://localhost:8080',
  {
    filter: 'downloading',
    sort: 'progress',
    reverse: true,
  });
});
```

6.2 Sync Main Data: /api/v2/sync/maindata

The sync API provides **incremental updates** for efficient real-time monitoring. Instead of polling the full torrent list, you request only changes since the last request.

Method: GET

Parameter	Type	Required	Description
rid	integer	optional	Response ID. If not provided or 0, returns full data. If different from server's last RID, returns full update.

Response Format

```
{
  "rid": 15,
  "full_update": false,
  "torrents": {
    "8c212779b4abde7c6bc608063a0d008b7e40ce32": {
      "state": "pausedUP",
      "progress": 1.0,
      "dlspeed": 0
    }
  },
  "torrents_removed": ["hash1", "hash2"],
  "categories": {
    "NewCategory": {"name": "NewCategory", "savePath": "/downloads/New"}
  },
  "categories_removed": ["OldCategory"],
  "tags": ["newTag"],
  "tags_removed": ["oldTag"],
  "server_state": {
    "dl_info_speed": 1048576,
    "up_info_speed": 524288,
    "dl_info_data": 1073741824,
    "up_info_data": 536870912
  }
}
```

Property	Type	Description
rid	integer	New response ID for next request
full_update	bool	Whether this contains all data (true) or just changes (false)
torrents	object	Changed torrents, keyed by hash (contains partial fields)
torrents_removed	array	Hashes of removed torrents
categories	object	Added/modified categories
categories_removed	array	Removed category names

Property	Type	Description
tags	array	Added tags
tags_removed	array	Removed tags
trackers	object	Added/modified trackers (v5.0+)
trackers_removed	array	Removed trackers (v5.0+)
server_state	object	Global transfer info

Note: torrents only includes fields that changed. You must maintain local state and merge updates. [6]

Sync Polling Strategy

```
class QBittorrentSyncPoller {
  constructor(baseUrl, intervalMs = 2000) {
    this.baseUrl = baseUrl;
    this.intervalMs = intervalMs;
    this.rid = 0;
    this.torrents = new Map();
    this.onUpdate = null;
    this.timer = null;
  }

  async poll() {
    try {
      const response = await fetch(
        `${this.baseUrl}/api/v2/sync/maindata?rid=${this.rid}`,
        {
          credentials: 'include',
          headers: { 'Referer': this.baseUrl },
        }
      );

      if (!response.ok) return;

      const data = await response.json();
      this.rid = data.rid;

      if (data.full_update) {
        // Reset all data
        this.torrents.clear();
      }

      // Apply torrent updates
      if (data.torrents) {
        for (const [hash, update] of
          Object.entries(data.torrents)) {
          const existing = this.torrents.get(hash) || {};
          this.torrents.set(hash, { ...existing, ...update });
        }
      }
    }
  }
}
```

```

    }
  }

  // Remove deleted torrents
  if (data.torrents_removed) {
    for (const hash of data.torrents_removed) {
      this.torrents.delete(hash);
    }
  }

  if (this.onUpdate) {
    this.onUpdate({
      torrents: Array.from(this.torrents.values()),
      serverState: data.server_state,
      categories: data.categories,
      tags: data.tags,
    });
  }
} catch (error) {
  console.error('Sync poll error:', error);
}
}

start() {
  this.poll();
  this.timer = setInterval(() => this.poll(), this.intervalMs);
}

stop() {
  if (this.timer) {
    clearInterval(this.timer);
    this.timer = null;
  }
}
}

```

6.3 Get Torrent Generic Properties: /api/v2/torrents/properties

Method: GET

Parameter	Type	Required	Description
hash	string	yes	Torrent hash

Response Fields

Property	Type	Description
save_path	string	Torrent save path
creation_date	integer	

Property	Type	Description
		Torrent creation date (Unix timestamp)
piece_size	integer	Piece size in bytes
comment	string	Torrent comment
total_wasted	integer	Total wasted data (bytes)
total_uploaded	integer	Total uploaded (bytes)
total_uploaded_session	integer	Uploaded this session (bytes)
total_downloaded	integer	Total downloaded (bytes)
total_downloaded_session	integer	Downloaded this session (bytes)
up_limit	integer	Upload limit (bytes/s)
dl_limit	integer	Download limit (bytes/s)
time_elapsed	integer	Elapsed time (seconds)
seeding_time	integer	Seeding time (seconds)
nb_connections	integer	Active connections
nb_connections_limit	integer	Connection limit
share_ratio	float	Share ratio
addition_date	integer	When torrent was added (Unix timestamp)
completion_date	integer	Completion date (Unix timestamp)
created_by	string	Torrent creator
dl_speed_avg	integer	Average download speed (bytes/s)
dl_speed	integer	Current download speed (bytes/s)
eta	integer	ETA (seconds)
last_seen	integer	Last seen complete (Unix timestamp)
peers	integer	Connected peers
peers_total	integer	Peers in swarm
pieces_have	integer	Pieces owned
pieces_num	integer	Total pieces
reannounce	integer	Seconds until reannounce
seeds	integer	Connected seeds
seeds_total	integer	Seeds in swarm
total_size	integer	Total torrent size (bytes)

6.4 Get Torrent Trackers: /api/v2/torrents/trackers

Method: GET

Parameter	Type	Required	Description
hash	string	yes	Torrent hash

Response Fields

Property	Type	Description
url	string	Tracker URL
status	integer	Tracker status (0=disabled, 1=not contacted, 2=working, 3=updating, 4=not working)
tier	integer	Priority tier
num_peers	integer	Peers reported by tracker
num_seeds	integer	Seeds reported by tracker
num_leeches	integer	Leeches reported by tracker
num_downloaded	integer	Completed downloads reported
msg	string	Tracker message

6.5 Get Torrent Contents (Files): /api/v2/torrents/files

Method: GET

Parameter	Type	Required	Description
hash	string	yes	Torrent hash
indexes	string	optional	Filter specific file indexes (comma-separated, since v2.8.2)

Response Fields

Property	Type	Description
index	integer	File index (since v2.8.2)
name	string	File path/name
size	integer	File size (bytes)
progress	float	

Property	Type	Description
		File download progress (0.0 to 1.0)
priority	integer	File priority (0=skip, 1=normal, 6=high, 7=maximal)
is_seed	bool	Whether file is complete
piece_range	array	Range of piece indices
availability	float	Piece availability

7. Torrent Management

7.1 Pause/Stop Torrents: /api/v2/torrents/stop (v5.0+) / /api/v2/torrents/pause (legacy)

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \ , or all

Pause specific torrents

```
curl -X POST http://localhost:8080/api/v2/torrents/stop \
  --cookie "SID=<your_sid>" \
  --data 'hashes=abc123...|def456...'
```

Pause all torrents

```
curl -X POST http://localhost:8080/api/v2/torrents/stop \
  --cookie "SID=<your_sid>" \
  --data 'hashes=all'
```

Version Note: In v4.x, the endpoint is /api/v2/torrents/pause. In v5.0+, it is /api/v2/torrents/stop. The pause alias still works in most v5.x versions. ^[121]

7.2 Resume/Start Torrents: /api/v2/torrents/start (v5.0+) / /api/v2/torrents/resume (legacy)

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \ , or all

Version Note: In v4.x, the endpoint is `/api/v2/torrents/resume`. In v5.0+, it is `/api/v2/torrents/start`. [¹²¹]

7.3 Delete Torrents: `/api/v2/torrents/delete`

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by <code>\ </code> , or <code>all</code>
deleteFiles	bool	yes (default: false)	Whether to delete downloaded files

Remove torrent but keep files

```
curl -X POST http://localhost:8080/api/v2/torrents/delete \
  --cookie "SID=<your_sid>" \
  --data 'hashes=abc123...' \
  --data 'deleteFiles=false'
```

Remove torrent and delete files

```
curl -X POST http://localhost:8080/api/v2/torrents/delete \
  --cookie "SID=<your_sid>" \
  --data 'hashes=abc123...' \
  --data 'deleteFiles=true'
```

Delete all torrents and their files

```
curl -X POST http://localhost:8080/api/v2/torrents/delete \
  --cookie "SID=<your_sid>" \
  --data 'hashes=all' \
  --data 'deleteFiles=true'
```

Default Change: In older versions, `deleteFiles` had to be explicitly specified. Since API v2.7/v4.3.2, it defaults to `false`.¹³

7.4 Recheck Torrents: `/api/v2/torrents/recheck`

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by <code>\ </code> , or <code>all</code>

7.5 Reannounce Torrents: `/api/v2/torrents/reannounce`

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \ , or all

7.6 Set Category: /api/v2/torrents/setCategory

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \ , or all
category	string	yes	Category name (empty string to remove category)

7.7 Set Tags: /api/v2/torrents/setTags (v5.1+) / /api/v2/torrents/addTags + /api/v2/torrents/removeTags (legacy)

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \ , or all
tags	string	yes	Comma-separated tags to set (replaces existing tags)

Set tags (replaces all existing tags)

```
curl -X POST http://localhost:8080/api/v2/torrents/setTags \
  --cookie "SID=<your_sid>" \
  --data 'hashes=abc123...' \
  --data 'tags=movies,hd'
```

Add tags (legacy, v4.x style)

```
curl -X POST http://localhost:8080/api/v2/torrents/addTags \
  --cookie "SID=<your_sid>" \
  --data 'hashes=abc123...' \
  --data 'tags=newtag'
```

Remove tags

```
curl -X POST http://localhost:8080/api/v2/torrents/removeTags \
  --cookie "SID=<your_sid>" \
  --data 'hashes=abc123...' \
  --data 'tags=oldtag'
```

v5.1 Change: torrents/setTags was added. Previously, use addTags/removeTags. ¹³

7.8 Set Automatic Torrent Management: /api/v2/torrents/setAutoManagement

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \, or all
enable	bool	yes	true to enable, false to disable

7.9 Toggle Sequential Download: /api/v2/torrents/toggleSequentialDownload

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \, or all

7.10 Set Force Start: /api/v2/torrents/setForceStart

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \, or all
value	bool	yes	true = force start, false = normal

7.11 Set Download/Upload Limits

Set Download Limit: POST /api/v2/torrents/setDownloadLimit

Set Upload Limit: POST /api/v2/torrents/setUploadLimit

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \, or all
limit	integer	yes	Limit in bytes/second (-1 or 0 = unlimited)

7.12 Set Share Limits: /api/v2/torrents/setShareLimits

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \, or all
ratioLimit	float	yes	Ratio limit (-2 = use global, -1 = unlimited)
seedingTimeLimit	integer	yes	Seeding time in minutes (-2 = use global, -1 = unlimited)

7.13 Set Torrent Location: /api/v2/torrents/setLocation

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \, or all
location	string	yes	New save path (must exist)

7.14 Rename Torrent: /api/v2/torrents/rename

Method: POST

Parameter	Type	Required	Description
hash	string	yes	Torrent hash
name	string	yes	New torrent name

7.15 Priority Control

Endpoint	Description
POST /api/v2/torrents/increasePrio?hashes=...	Increase priority
POST /api/v2/torrents/decreasePrio?hashes=...	Decrease priority
POST /api/v2/torrents/topPrio?hashes=...	Set top priority
POST /api/v2/torrents/bottomPrio?hashes=...	Set bottom priority

Error: Returns 409 if torrent queueing is not enabled. [6]

7.16 Set File Priority: /api/v2/torrents/filePrio

Method: POST

Parameter	Type	Required	Description
hash	string	yes	Torrent hash
id	string	yes	File IDs (comma-separated) or * for all
priority	integer	yes	0=do not download, 1=normal, 6=high, 7=maximal

8. Application Preferences

8.1 Get Preferences: /api/v2/app/preferences

Method: GET

Returns a JSON object with all application settings. Key preferences for integration:

Property	Type	Description
save_path	string	Default save path
temp_path_enabled	bool	Whether incomplete folder is used
temp_path	string	Path for incomplete torrents
auto_tmm_enabled	bool	Whether Auto TMM is enabled by default
start_paused_enabled	bool	Whether torrents start paused
max_active_downloads	integer	Max simultaneous downloads
max_active_torrents	integer	Max active torrents
max_active_uploads	integer	Max simultaneous uploads
dl_limit	integer	Global download limit (KiB/s, -1 = no limit)

Property	Type	Description
up_limit	integer	Global upload limit (KiB/s, -1 = no limit)
alt_dl_limit	integer	Alternative download limit (KiB/s)
alt_up_limit	integer	Alternative upload limit (KiB/s)
scheduler_enabled	bool	Whether speed limit scheduler is active
dht	bool	DHT enabled
pex	bool	PeX enabled
lsd	bool	LSD enabled
encryption	integer	0=prefer, 1=force on, 2=force off
queueing_enabled	bool	Whether torrent queueing is enabled
web_ui_username	string	WebUI username
web_ui_password	string	WebUI password (write-only since v2.3.0)
web_ui_csrf_protection_enabled	bool	CSRF protection enabled
web_ui_max_auth_fail_count	integer	Max failed login attempts before ban
web_ui_ban_duration	integer	Login ban duration (seconds)
locale	string	UI language (e.g., en)
rss_refresh_interval	integer	RSS refresh interval (minutes)
rss_max_articles_per_feed	integer	Max RSS articles per feed
rss_processing_enabled	bool	RSS processing enabled
rss_auto_downloading_enabled	bool	RSS auto-downloading enabled
add_trackers_enabled	bool	Auto-add trackers to new torrents
add_trackers	string	Tracker URLs to auto-add (newline-separated)
max_ratio_enabled	bool	

Property	Type	Description
		Whether global share ratio limit is enabled
max_ratio	float	Global share ratio limit
max_seeding_time_enabled	bool	Whether max seeding time is enabled
max_seeding_time	integer	Max seeding time (minutes)
max_ratio_act	integer	Action when ratio reached: 0=pause, 1=remove, 2=enable super seeding

8.2 Set Preferences: /api/v2/app/setPreferences

Method: POST

Parameter	Type	Required	Description
json	string	yes	JSON string of preferences to set (key-value pairs)

Important: Only include preferences you want to change. String values must be quoted; integer and boolean values must NOT be quoted. ^[6]

Set download path

```
curl -X POST http://localhost:8080/api/v2/app/setPreferences \
  --cookie "SID=<your_sid>" \
  --data 'json={"save_path":"/
    downloads","queueing_enabled":false}'
```

Set global speed limits (in KiB/s)

```
curl -X POST http://localhost:8080/api/v2/app/setPreferences \
  --cookie "SID=<your_sid>" \
  --data 'json={"dl_limit":10240,"up_limit":5120}'
```

Disable CSRF protection (for API-only access behind reverse proxy)

```
curl -X POST http://localhost:8080/api/v2/app/setPreferences \
  --cookie "SID=<your_sid>" \
  --data 'json={"web_ui_csrf_protection_enabled":false}'
```

8.3 Get Default Save Path: /api/v2/app/defaultSavePath

Method: GET

Returns: String with the default save path (e.g., /home/user/Downloads)

8.4 Application Version Endpoints

Endpoint	Method	Returns
/api/v2/app/version	GET	Application version string (e.g., v5.0.0)
/api/v2/app/webapiVersion	GET	Web API version string (e.g., 2.11.0)
/api/v2/app/buildInfo	GET	JSON with qt, libtorrent, boost, openssl, bitness

Check versions

```
curl http://localhost:8080/api/v2/app/version --cookie "SID=<sid>"  
# v5.0.1
```

```
curl http://localhost:8080/api/v2/app/webapiVersion --cookie  
"SID=<sid>"  
# 2.11.0
```

9. Category Management

9.1 Get All Categories: /api/v2/torrents/categories

Method: GET

Returns JSON object where each key is a category name:

```
{  
  "Movies": {  
    "name": "Movies",  
    "savePath": "/downloads/Movies"  
  },  
  "TV": {  
    "name": "TV",  
    "savePath": "/downloads/TV"  
  }  
}
```

9.2 Create Category: /api/v2/torrents/createCategory

Method: POST

Parameter	Type	Required	Description
category	string	yes	Category name
savePath	string	optional	Save path for this category

```
curl -X POST http://localhost:8080/api/v2/torrents/createCategory \
  --cookie "SID=<your_sid>" \
  --data 'category=Movies' \
  --data 'savePath=/downloads/Movies'
```

9.3 Edit Category: /api/v2/torrents/editCategory

Method: POST

Parameter	Type	Required	Description
category	string	yes	Category name
savePath	string	yes	New save path

```
curl -X POST http://localhost:8080/api/v2/torrents/editCategory \
  --cookie "SID=<your_sid>" \
  --data 'category=Movies' \
  --data 'savePath=/media/Movies'
```

9.4 Remove Categories: /api/v2/torrents/removeCategories

Method: POST

Parameter	Type	Required	Description
categories	string	yes	Category names separated by \n (newline)

```
# Remove multiple categories
curl -X POST http://localhost:8080/api/v2/torrents/
  removeCategories \
  --cookie "SID=<your_sid>" \
  --data '$'categories=Category1\nCategory2'
```

10. Tag Management

10.1 Get All Tags: /api/v2/torrents/tags

Method: GET

Returns: JSON array of tag strings

```
["movies", "tv", "hd", "sd"]
```

10.2 Create Tags: /api/v2/torrents/createTags

Method: POST

Parameter	Type	Required	Description
tags	string	yes	Tag names separated by ,

```
curl -X POST http://localhost:8080/api/v2/torrents/createTags \
  --cookie "SID=<your_sid>" \
  --data 'tags=movies,tv,hd'
```

10.3 Delete Tags: /api/v2/torrents/deleteTags

Method: POST

Parameter	Type	Required	Description
tags	string	yes	Tag names separated by ,

10.4 Add Tags to Torrents: /api/v2/torrents/addTags

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \ , or all
tags	string	yes	Tag names separated by ,

10.5 Remove Tags from Torrents: /api/v2/torrents/removeTags

Method: POST

Parameter	Type	Required	Description
hashes	string	yes	Hash(es) separated by \ , or all

Parameter	Type	Required	Description
tags	string	yes	Tag names separated by ,

11. Search API

The Search API allows programmatic searching via qBittorrent's search plugins.

11.1 Start Search: /api/v2/search/start

Method: POST

Parameter	Type	Required	Description
pattern	string	yes	Search term (e.g., Ubuntu 22.04)
plugins	string	yes	Plugin names separated by \, , or all, or enabled
category	string	yes	Search category (plugin-specific), or all

Returns: {"id": 12345} (search job ID)

HTTP Status	Scenario
409	Max concurrent searches reached (limit: 5)
200	Success

```
# Start a search
curl -X POST http://localhost:8080/api/v2/search/start \
  --cookie "SID=<your_sid>" \
  --data 'pattern=Ubuntu' \
  --data 'plugins=all' \
  --data 'category=all'
# {"id": 1}
```

11.2 Stop Search: /api/v2/search/stop

Method: POST

Parameter	Type	Required	Description
id	integer	yes	Search job ID

11.3 Get Search Status: /api/v2/search/status

Method: GET

Parameter	Type	Required	Description
id	integer	optional	Specific search job ID (omit for all)

Returns: Array of search job status objects:

```
[
  {
    "id": 1,
    "status": "Running",
    "total": 150
  }
]
```

Status values: Running, Stopped

11.4 Get Search Results: /api/v2/search/results

Method: GET

Parameter	Type	Required	Description
id	integer	yes	Search job ID
limit	integer	optional	Max results to return
offset	integer	optional	Result offset

Returns:

```
{
  "results": [
    {
      "descrLink": "https://example.com/torrent/123",
      "fileName": "Ubuntu 22.04 ISO",
      "fileSize": 3825205248,
      "fileUrl": "magnet:?xt=urn:btih:...",
      "nbLeechers": 5,
      "nbSeeders": 150,
      "siteUrl": "https://example.com",
      "torrentUrl": "https://example.com/download/123.torrent"
    }
  ],
  "status": "Running"
}
```

11.5 Delete Search: /api/v2/search/delete

Method: POST

Parameter	Type	Required	Description
id	integer	yes	Search job ID

11.6 Manage Search Plugins

Endpoint	Method	Description
/api/v2/search/plugins	GET	List all plugins with their status
/api/v2/search/installPlugin	POST	Install plugin from URL(s)
/api/v2/search/uninstallPlugin	POST	Uninstall plugin(s) by name
/api/v2/search/enablePlugin	POST	Enable/disable plugin
/api/v2/search/updatePlugins	POST	Update all plugins

Plugin enable/disable:

```
curl -X POST http://localhost:8080/api/v2/search/enablePlugin \
  --cookie "SID=<your_sid>" \
  --data 'names=1337x,eztv' \
  --data 'enable=true'
```

[Source: GitHub Wiki - WebUI API “Search” section] ^[6] ^[39]

12. RSS API

| **Status:** Marked as experimental in API docs.

12.1 Add Folder: /api/v2/rss/addFolder

Method: POST

Parameter	Type	Required	Description
path	string	yes	Full folder path (e.g., The Pirate Bay\\Top100)

Returns: 200 on success, 409 on failure

```
curl -X POST http://localhost:8080/api/v2/rss/addFolder \
  --cookie "SID=<your_sid>" \
  --data 'path=TV Shows'
```

12.2 Add Feed: /api/v2/rss/addFeed

Method: POST

Parameter	Type	Required	Description
url	string	yes	RSS feed URL
path	string	optional	Full path (e.g., TV Shows\\EZTV)

12.3 Remove Item: /api/v2/rss/removeItem

Method: POST

Parameter	Type	Required	Description
path	string	yes	Full path to folder or feed

12.4 Move Item: /api/v2/rss/moveItem

Method: POST

Parameter	Type	Required	Description
itemPath	string	yes	Current full path
destPath	string	yes	New full path

12.5 Get All Items: /api/v2/rss/items

Method: POST

Parameter	Type	Required	Description
withData	bool	optional	Include feed articles

Returns nested JSON of folders and feeds:

```
{
  "RSS Feed Name": {
    "url": "https://example.com/feed.xml",
    "uid": "{...}",
    "articles": [
      {
        "date": "2024-01-15T10:30:00",
        "id": "article-id",
        "isRead": false,
        "title": "Article Title",

```



```

        "torrentURL": "https://example.com/download.torrent"
    }
}
}
}

```

12.6 Mark as Read: /api/v2/rss/markAsRead

Method: POST

Parameter	Type	Required	Description
itemPath	string	yes	Path to feed
articleId	string	optional	Specific article ID (omit to mark entire feed as read)

12.7 Refresh Item: /api/v2/rss/refreshItem

Method: POST

Parameter	Type	Required	Description
itemPath	string	yes	Path to feed

12.8 Auto-Downloading Rules

Endpoint	Method	Description
/api/v2/rss/setRule	POST	Create/set auto-downloading rule
/api/v2/rss/renameRule	POST	Rename a rule
/api/v2/rss/removeRule	POST	Remove a rule
/api/v2/rss/rules	GET	Get all rules
/api/v2/rss/ matchingArticles	GET	Get articles matching a rule

Rule definition (POST to setRule):

```

{
  "enabled": true,
  "mustContain": "1080p",
  "mustNotContain": "cam",
  "useRegex": false,
  "episodeFilter": "",
  "smartFilter": false,
  "previouslyMatchedEpisodes": [],
  "affectedFeeds": ["https://example.com/feed.xml"],
  "savePath": "/downloads/Movies",

```

```
"assignedCategory": "Movies",  
"lastMatch": "",  
"addPaused": false,  
"torrentContentLayout": "Original"  
}
```

[Source: GitHub Wiki - WebUI API “RSS” section] ^[6] ^[59]

13. Log API

13.1 Get Main Log: /api/v2/log/main

Method: GET

Parameter	Type	Required	Description
normal	bool	optional	Include normal messages (default: true)
info	bool	optional	Include info messages (default: true)
warning	bool	optional	Include warning messages (default: true)
critical	bool	optional	Include critical messages (default: true)
last_known_id	integer	optional	Only return entries with ID > this value (default: -1 for all)

Returns: JSON array of log entries

Property	Type	Description
id	integer	Message ID
message	string	Log message text
timestamp	integer	Unix timestamp (seconds since epoch)
type	integer	1=Normal, 2=Info, 4=Warning, 8=Critical

Timestamp Change: Before v4.5.0, timestamps were in **milliseconds**. Since v4.5.0, they are in **seconds**. ^[6]

```
# Get all log entries  
curl "http://localhost:8080/api/v2/log/main?last_known_id=-1" \
```

```
--cookie "SID=<your_sid>"

# Get only warnings and critical
curl "http://localhost:8080/api/v2/log/main?
      normal=false&info=false&last_known_id=-1" \
--cookie "SID=<your_sid>"
```

13.2 Get Peer Log: /api/v2/log/peers

Method: GET

Parameter	Type	Required	Description
last_known_id	integer	optional	Only return entries with ID > this value

Returns: JSON array of peer log entries

Property	Type	Description
id	integer	Entry ID
ip	string	Peer IP address
timestamp	integer	Unix timestamp (seconds)
blocked	bool	Whether peer was blocked
reason	string	Reason for block

14. Error Handling

14.1 HTTP Status Codes

Status Code	Meaning	Common Causes
200	OK	Request successful
400	Bad Request	Missing required parameters, invalid argument format
401	Unauthorized	XSS or host header validation failure
403	Forbidden	Not logged in, IP banned, or insufficient permissions
404	Not Found	Torrent hash not found
405	Method Not Allowed	Wrong HTTP method (GET vs POST) since v4.4.4

Status Code	Meaning	Common Causes
409	Conflict	Operation cannot be performed (e.g., queueing not enabled, category doesn't exist)
415	Unsupported Media Type	Invalid torrent file data
500	Internal Server Error	Server error

14.2 Common Error Scenarios

```

async function handleQBittorrentError(response) {
  switch (response.status) {
    case 403:
      // Session expired or IP banned
      console.error('Authentication failed - need to re-login');
      // Clear session and redirect to login
      break;

    case 404:
      console.error('Torrent not found');
      break;

    case 409:
      const body = await response.text();
      if (body.includes('queue')) {
        console.error('Torrent queueing is not enabled');
      } else if (body.includes('category')) {
        console.error('Category does not exist');
      }
      break;

    case 415:
      console.error('Invalid torrent file or magnet link');
      break;

    case 400:
      console.error('Bad request - check parameters');
      break;

    default:
      console.error(`Unexpected error: ${response.status}`);
  }
}

```

14.3 Session Management Strategy

```
class QBittorrentSessionManager {
  constructor(baseUrl) {
    this.baseUrl = baseUrl;
    this.isAuthenticated = false;
  }

  async ensureAuthenticated() {
    // Try a lightweight API call to check session
    try {
      const response = await fetch(`${this.baseUrl}/api/v2/app/
        version`, {
        credentials: 'include',
        headers: { 'Referer': this.baseUrl },
      });

      if (response.ok) {
        this.isAuthenticated = true;
        return true;
      }

      if (response.status === 403 || response.status === 401) {
        this.isAuthenticated = false;
        return false;
      }
    } catch (error) {
      console.error('Connection error:', error);
      return false;
    }
  }

  async withAuth(apiCall) {
    if (!this.isAuthenticated) {
      const authed = await this.ensureAuthenticated();
      if (!authed) {
        throw new Error('Not authenticated');
      }
    }

    try {
      return await apiCall();
    } catch (error) {
      if (error.status === 403) {
        this.isAuthenticated = false;
        // Could trigger re-auth here
      }
      throw error;
    }
  }
}
```

14.4 Rate Limiting Considerations

qBittorrent does not have explicit API rate limiting, but excessive polling can cause performance issues:

- /sync/maindata is designed for incremental polling (recommended: 1-5 second intervals)
- /torrents/info returns full data and can be slow with many torrents (>1000)
- One user reported /sync/maindata taking ~60 seconds with ~4000 torrents on an Atom CPU [⁴⁹]
- Recommend polling intervals of 2-5 seconds for the sync endpoint
- For the browser extension use case (adding torrents), no polling is needed - just POST to /torrents/add

15. Version Differences (4.x vs 5.x)

15.1 Breaking Changes Summary

Feature	v4.x	v5.0+
Torrent states	pausedUP, pausedDL	stoppedUP, stoppedDL (old names still accepted)
Pause endpoint	/torrents/pause	/torrents/stop (alias still works)
Resume endpoint	/torrents/resume	/torrents/start (alias still works)
Filter values	paused	stopped (new running filter added)
Add param paused	Works	Still works; is_stopped is the new parameter
Add param cookie	Supported	Removed in API v2.11.3 (use app/ cookies instead)

15.2 New in v5.0+

Feature	API Version	Description
isPrivate field	v5.0.0	Indicates private tracker torrent

Feature	API Version	Description
reannounce field	v2.9.3	Time until next tracker reannounce
downloadPath param	v5.0.0	Path for incomplete torrents
stopCondition param	v5.0.0	Stop at MetadataReceived or FilesChecked
is_stopped param	v2.11.0	Replaces paused for adding stopped torrents
forced param	v2.11.0	Add torrent in forced state
torrents/setTags	v5.1.0	Set/replace all tags at once
app/cookies + app/setCookies	v2.11.3	Cookie management endpoints
API Key Authentication	v2.14.1 (v5.2.0)	Bearer token auth alternative

15.3 API Version Changelog Summary

API Version	Key Changes
v2.0	New path structure /api/v2/...
v2.0.1	Added hashes to /torrents/info
v2.1.0	savePath in createCategory; editCategory added
v2.1.1	/torrents/categories; /search/endpoints
v2.2.0	editTracker, removeTracker, addTags, removeTags, etc.
v2.3.0	/app/buildInfo; /torrents/addPeers; /transfer/banPeers; tag management
v2.4.0	/torrents/renameFile
v2.4.1	stalled, stalled_uploading, stalled_downloading filters
v2.5.0	RSS improvements
v2.6.0	search/categories removed; search plugin improvements
v2.7.0	content_layout param (supersedes root_folder)
v2.8.0	ssl_certificate, ssl_private_key, ssl_dh_params

API Version	Key Changes
v2.8.1	ratioLimit, seedingTimeLimit params for /torrents/add
v2.8.2	indexes param for /torrents/ files
v2.8.3	tag filter for /torrents/info
v2.9.3	reannounce field in /torrents/ info
v2.11.0	is_stopped and forced params; state renames (pausedUP- >stoppedUP)
v2.11.3	app/cookies, app/setCookies; removed cookie from /torrents/ add
v2.14.1	API Key Authentication (Bearer token)

[Sources: GitHub Wiki - WebUI API Changes sections for both v4.1 and v5.0] ^[6] ^[121]

16. Python qbittorrent-api Library

The qbittorrent-api Python library (formerly python-qbittorrent) provides a comprehensive Python interface to the WebUI API.

16.1 Installation

```
pip install qbittorrent-api
```

16.2 Authentication

```
import qbittorrentapi
```

```
# Method 1: Auto-authentication on first API call
```

```
client = qbittorrentapi.Client(
    host='localhost',
    port=8080,
    username='admin',
    password='adminadmin'
)
```

```
# Method 2: Explicit login
```

```
try:
    client.auth_log_in()
except qbittorrentapi.LoginFailed as e:
```



```

    print(f"Login failed: {e}")

# Check versions
print(f"qBittorrent: {client.app.version}")
print(f"Web API: {client.app.web_api_version}")

# The library auto-refreshes cookies when they expire
# No manual session management needed

```

16.3 Adding Torrents

```

import qbittorrentapi

client = qbittorrentapi.Client(host='localhost:8080',
                               username='admin', password='adminadmin')

# Add magnet link
client.torrents_add(
    urls='magnet:?
        xt=urn:btih:08ada5a7a6183aae1e09d831df6748d566095a10&dn=Sintel',
    category='Movies',
    save_path='/downloads/Movies',
    is_paused=False,
    tags='animation,hd'
)

# Add multiple magnet links
magnets = '\n'.join([
    'magnet:?xt=urn:btih:hash1&dn=Movie1',
    'magnet:?xt=urn:btih:hash2&dn=Movie2',
])
client.torrents_add(urls=magnets, category='Batch')

# Upload .torrent file
with open('/path/to/file.torrent', 'rb') as f:
    client.torrents_add(
        torrent_files=f,
        category='TV',
        save_path='/downloads/TV',
        tags='tv-series'
    )

# Add with speed limits and ratio limit
client.torrents_add(
    urls='magnet:?xt=urn:btih:...',
    dl_limit=1048576,      # 1 MB/s download limit
    up_limit=524288,      # 512 KB/s upload limit
    ratio_limit=2.0,      # Stop seeding at 2.0 ratio
    seeding_time_limit=10080, # 1 week max seeding
    sequential_download=True,

```

```
    first_last_piece_prio=True,  
)
```

16.4 Monitoring Torrents

```
# Get all torrents
```

```
torrents = client.torrents_info()  
for torrent in torrents:  
    print(f"{torrent.name}: {torrent.state}  
          ({torrent.progress*100:.1f}%)")
```

```
# Filter by state
```

```
downloading = client.torrents_info(status_filter='downloading')  
completed = client.torrents_info(status_filter='completed')
```

```
# Filter by category
```

```
movies = client.torrents_info(category='Movies')
```

```
# Filter by tag
```

```
tagged = client.torrents_info(tag='hd')  # v2.8.3+
```

```
# Sort and paginate
```

```
page = client.torrents_info(  
    status_filter='all',  
    sort='added_on',  
    reverse=True,  
    limit=50,  
    offset=0  
)
```

```
# Get specific torrent by hash
```

```
torrent = client.torrents_info(torrent_hashes='abc123...')
```

```
# Get detailed properties
```

```
props = client.torrents_properties(torrent_hash='abc123...')  
print(f"Save path: {props.save_path}")  
print(f"Share ratio: {props.share_ratio}")  
print(f"ETA: {props.eta} seconds")
```

```
# Get trackers
```

```
trackers = client.torrents_trackers(torrent_hash='abc123...')  
for tracker in trackers:  
    print(f"{tracker.url}: {tracker.status}")
```

```
# Get files
```

```
files = client.torrents_files(torrent_hash='abc123...')  
for f in files:  
    print(f"{f.name}: {f.progress*100:.1f}%)")
```

16.5 Managing Torrents

```
# Pause/stop
torrent_hash = 'abc123...'
client.torrents_stop(torrent_hashes=torrent_hash)
client.torrents_stop(torrent_hashes='all') # Stop all

# Resume/start
client.torrents_start(torrent_hashes=torrent_hash)
client.torrents_start(torrent_hashes='all')

# Delete (keep files)
client.torrents_delete(delete_files=False,
                       torrent_hashes=torrent_hash)

# Delete (remove files too)
client.torrents_delete(delete_files=True,
                       torrent_hashes=torrent_hash)

# Recheck
client.torrents_recheck(torrent_hashes=torrent_hash)

# Set category
client.torrents_set_category(category='NewCategory',
                              torrent_hashes=torrent_hash)

# Set tags (v5.1+)
client.torrents_set_tags(tags='tag1,tag2',
                          torrent_hashes=torrent_hash)

# Add tags
client.torrents_add_tags(tags='newtag',
                          torrent_hashes=torrent_hash)

# Remove tags
client.torrents_remove_tags(tags='oldtag',
                             torrent_hashes=torrent_hash)

# Set speed limits
client.torrents_set_download_limit(limit=1048576,
                                    torrent_hashes=torrent_hash)
client.torrents_set_upload_limit(limit=512000,
                                  torrent_hashes=torrent_hash)

# Set share limits
client.torrents_set_share_limits(
    ratio_limit=2.0,
    seeding_time_limit=10080,
    torrent_hashes=torrent_hash
)

# Set location
```

```

client.torrents_set_location(location='/new/path',
                             torrent_hashes=torrent_hash)

# Force start
client.torrents_set_force_start(enable=True,
                                 torrent_hashes=torrent_hash)

# Rename
client.torrents_rename(torrent_hash=torrent_hash,
                       new_torrent_name='New Name')

# Auto TMM
client.torrents_set_auto_management(enable=True,
                                    torrent_hashes=torrent_hash)

# Sequential download toggle
client.torrents_toggle_sequential_download(torrent_hashes=torrent_hash)

# Priority
client.torrents_top_priority(torrent_hashes=torrent_hash)
client.torrents_bottom_priority(torrent_hashes=torrent_hash)
client.torrents_increase_priority(torrent_hashes=torrent_hash)
client.torrents_decrease_priority(torrent_hashes=torrent_hash)

```

16.6 Categories and Tags

```

# Categories
client.torrents_create_category(name='Movies', save_path='/
downloads/Movies')
client.torrents_edit_category(name='Movies', save_path='/media/
Movies')
categories = client.torrents_categories()
client.torrents_remove_categories(categories='OldCategory')

# Tags
client.torrents_create_tags(tags='movies,tv,hd,sd')
tags = client.torrents_tags()
client.torrents_delete_tags(tags='oldtag')

```

16.7 Sync / Incremental Updates

```

# Full data
maindata = client.sync_maindata(rid=0)
print(f"Full update: {maindata.full_update}")
print(f"Torrents: {len(maindata.torrents)}")

# Incremental - get changes
rid = maindata.rid
changes = client.sync_maindata(rid=rid)
print(f"Changes only: {not changes.full_update}")

# Using the delta helper (auto-tracks RID)

```

```
md = client.sync.maindata.delta()
# ... time passes ...
md2 = client.sync.maindata.delta()
# Gets only changes since last call
```

16.8 Transfer Info

```
# Global info
info = client.transfer_info()
print(f"Download: {info.dl_info_speed} bytes/s")
print(f"Upload: {info.up_info_speed} bytes/s")
print(f"DHT nodes: {info.dht_nodes}")

# Speed limits
client.transfer_set_download_limit(limit=1048576) # 1 MB/s
client.transfer_set_upload_limit(limit=512000)    # 512 KB/s

# Alt speed mode
client.transfer_set_speed_limits_mode(intended_state=True)
```

16.9 Search

```
# Start search
job = client.search_start(pattern='Ubuntu', plugins='all',
                           category='all')
print(f"Search ID: {job.id}")

# Poll until complete
import time
while True:
    status = client.search_status(search_id=job.id)
    if status[0].status == 'Stopped':
        break
    time.sleep(1)

# Get results
results = client.search_results(search_id=job.id, limit=100,
                                offset=0)
for result in results.results:
    print(f"{result.fileName}: {result.nbSeeders} seeds")

# Cleanup
client.search_delete(search_id=job.id)
```

16.10 Error Handling

```
from qbittorrentapi import (
    Client, LoginFailed, Forbidden403Error, NotFound404Error,
    Conflict409Error, HTTP415Error, HTTP400Error
)
```

```

try:
    client = Client(host='localhost:8080', username='admin',
                    password='wrong')
    client.auth_log_in()
except LoginFailed as e:
    print(f"Login failed: {e}")

try:
    client.torrents_properties(torrent_hash='nonexistent')
except NotFound404Error:
    print("Torrent not found")

try:
    client.torrents_increase_priority(torrent_hashes='all')
except Conflict409Error:
    print("Queueing is not enabled")

try:
    client.torrents_add(urls='invalid-data')
except HTTP415Error:
    print("Invalid torrent data")

```

[Source: qbittorrent-api PyPI and ReadTheDocs] [13](#) [50](#) [\[62\]](#)

17. WebSocket/SSE and Real-Time Updates

17.1 No Native WebSocket/SSE Support

qBittorrent's WebUI API **does not provide WebSocket or Server-Sent Events (SSE)** endpoints for real-time updates. All real-time data must be obtained through polling.

17.2 Recommended Polling Strategy

For browser extensions, the recommended approach is:

1. **For “add torrent” functionality:** No polling needed. Just POST to `/torrents/add`.
2. **For status display:** Use `/sync/maindata` with incremental RID:

```

class TorrentPoller {
    constructor(baseUrl, onUpdate, intervalMs = 3000) {
        this.baseUrl = baseUrl;
        this.onUpdate = onUpdate;
        this.intervalMs = intervalMs;
    }
}

```

```

    this.rid = 0;
    this.timer = null;
    this.torrentMap = new Map();
  }

  async fetch() {
    try {
      const resp = await fetch(
        `${this.baseUrl}/api/v2/sync/maindata?rid=${this.rid}`,
        { credentials: 'include', headers: { 'Referer':
          this.baseUrl } }
      );
      if (!resp.ok) return;
      const data = await resp.json();
      this.rid = data.rid;

      if (data.full_update) this.torrentMap.clear();

      if (data.torrents) {
        for (const [hash, t] of Object.entries(data.torrents)) {
          const existing = this.torrentMap.get(hash) || {};
          this.torrentMap.set(hash, { ...existing, ...t });
        }
      }
      if (data.torrents_removed) {
        for (const h of data.torrents_removed)
          this.torrentMap.delete(h);
      }

      this.onUpdate({
        torrents: Array.from(this.torrentMap.values()),
        serverState: data.server_state,
      });
    } catch (e) {
      console.error('Poll error:', e);
    }
  }

  start() {
    this.fetch();
    this.timer = setInterval(() => this.fetch(), this.intervalMs);
  }

  stop() {
    if (this.timer) clearInterval(this.timer);
  }
}

```

17.3 Alternative: Direct Polling of /torrents/info

For simpler use cases, poll /torrents/info directly:

```
// Lightweight - only when needed
async function checkTorrentStatus(hash) {
  const resp = await fetch(
    `${baseUrl}/api/v2/torrents/info?hashes=${hash}`,
    { credentials: 'include', headers: { 'Referer': baseUrl } }
  );
  const torrents = await resp.json();
  return torrents[0]; // null if not found
}
```

18. Complete Working Examples

18.1 Complete Browser Extension: Add Magnet Link

```
// content_script.js - Intercept magnet link clicks

document.addEventListener('click', async (e) => {
  const link = e.target.closest('a[href^="magnet:"]');
  if (!link) return;

  e.preventDefault();
  const magnetUri = link.href;

  // Send to background script
  chrome.runtime.sendMessage({
    type: 'ADD_MAGNET',
    magnetUri,
    options: {
      category: detectCategory(link), // Infer from page context
    }
  }, (response) => {
    if (response.success) {
      showNotification('Torrent added to qBittorrent');
    } else {
      showNotification(`Failed: ${response.error}`, 'error');
    }
  });
});

// background.js - Service worker
chrome.runtime.onMessage.addListener((msg, sender, sendResponse)
  => {
    if (msg.type === 'ADD_MAGNET') {
      handleAddMagnet(msg.magnetUri, msg.options)
        .then(r => sendResponse({ success: true }))
        .catch(e => sendResponse({ success: false, error:
          e.message }));
      return true; // Async
    }
  });
```



```

    }
  });

  async function handleAddMagnet(magnetUri, options = {}) {
    const { qbUrl, qbUser, qbPass, defaultCategory } =
      await chrome.storage.sync.get(['qbUrl', 'qbUser', 'qbPass',
        'defaultCategory']);

    // Login
    const login = new URLSearchParams({ username: qbUser, password:
      qbPass });
    const loginRes = await fetch(`${qbUrl}/api/v2/auth/login`, {
      method: 'POST',
      headers: { 'Referer': qbUrl, 'Content-Type': 'application/x-
        www-form-urlencoded' },
      body: login.toString(),
      credentials: 'include',
    });
    if (!loginRes.ok) throw new Error('Login failed');

    // Add torrent
    const form = new FormData();
    form.append('urls', magnetUri);
    form.append('category', options.category || defaultCategory ||
      '');
    if (options.savePath) form.append('savepath', options.savePath);

    const addRes = await fetch(`${qbUrl}/api/v2/torrents/add`, {
      method: 'POST',
      credentials: 'include',
      headers: { 'Referer': qbUrl },
      body: form,
    });

    if (addRes.status === 415) throw new Error('Invalid magnet
      link');
    if (!addRes.ok) throw new Error(`Add failed: ${addRes.status}`);

    return true;
  }
}

```

18.2 Complete Python Script: Monitor and Manage

```

#!/usr/bin/env python3
"""Complete qBittorrent API management script"""

import qbittorrentapi
import time
import sys

# Connect
client = qbittorrentapi.Client(

```

```

        host='localhost:8080',
        username='admin',
        password='adminadmin'
    )

    try:
        client.auth_log_in()
    except qbittorrentapi.LoginFailed:
        print("Authentication failed")
        sys.exit(1)

    print(f"Connected to qBittorrent {client.app.version}")
    print(f"Web API version: {client.app.web_api_version}")

    # 1. Add a magnet link
    magnet = "magnet:?
                xt=urn:btih:08ada5a7a6183aae1e09d831df6748d566095a10&dn=Sintel"
    client.torrents_add(
        urls=magnet,
        category='Animation',
        save_path='/downloads/Movies',
        tags='hd,short-film'
    )
    print("Magnet link added")

    # 2. Wait and check status
    print("Waiting for metadata...")
    for _ in range(30):
        time.sleep(2)
        torrents = client.torrents_info(category='Animation')
        if torrents:
            t = torrents[0]
            print(f" {t.name}: {t.state} ({t.progress*100:.1f}%) - "
                  f"{t.num_seeds} seeds / {t.dlspeed/1024:.1f} KB/s")
            if t.state not in ('metaDL', 'checkingDL', 'downloading'):
                break

    # 3. List all active downloads
    print("\nActive downloads:")
    for t in client.torrents_info(status_filter='downloading'):
        print(f" {t.name}: {t.progress*100:.1f}% - "
              f"{t.dlspeed/1024/1024:.2f} MB/s down, "
              f"{t.upspeed/1024/1024:.2f} MB/s up")

    # 4. Set category on completed torrents
    for t in client.torrents_info(status_filter='completed'):
        if not t.category:
            client.torrents_set_category(category='Unsorted',
                                          torrent_hashes=t.hash)

    # 5. Apply tags to untagged torrents
    for t in client.torrents_info():

```

```

    if not t.tags:
        if 'movie' in t.name.lower():
            client.torrents_add_tags(tags='movie',
            torrent_hashes=t.hash)
        elif 'tv' in t.name.lower() or 's01e' in t.name.lower():
            client.torrents_add_tags(tags='tv',
            torrent_hashes=t.hash)

# 6. Clean up old completed torrents
for t in client.torrents_info(status_filter='completed'):
    if t.seeding_time > 7 * 24 * 60 * 60: # 7 days
        print(f"Removing old torrent: {t.name}")
        client.torrents_delete(delete_files=False,
        torrent_hashes=t.hash)

# Logout
client.auth_log_out()
print("Done!")

```

18.3 curl Reference Card

```

# ===== AUTHENTICATION =====
# Login (capture SID cookie)
curl -i -H 'Referer: http://localhost:8080' \
    -d 'username=admin&password=adminadmin' \
    http://localhost:8080/api/v2/auth/login

# Logout
curl -X POST http://localhost:8080/api/v2/auth/logout \
    --cookie "SID=<sid>"

# ===== ADDING TORRENTS =====
# Add magnet
curl -X POST http://localhost:8080/api/v2/torrents/add \
    --cookie "SID=<sid>" -d 'urls=magnet:?xt=urn:btih:...'

# Add from URL
curl -X POST http://localhost:8080/api/v2/torrents/add \
    --cookie "SID=<sid>" -d 'urls=https://example.com/file.torrent'

# Upload .torrent file
curl -X POST http://localhost:8080/api/v2/torrents/add \
    --cookie "SID=<sid>" -F "torrents=@file.torrent" -F
    "category=Movies"

# Add with options
curl -X POST http://localhost:8080/api/v2/torrents/add \
    --cookie "SID=<sid>" \
    -d 'urls=magnet:...' \
    -d 'savepath=/downloads/Movies' \
    -d 'category=Movies' \
    -d 'paused=true' \

```

```

    -d 'autoTMM=true' \
    -d 'sequentialDownload=true' \
    -d 'dlLimit=1048576' \
    -d 'upLimit=524288'

# ===== MONITORING =====
# List all torrents
curl http://localhost:8080/api/v2/torrents/info --cookie
    "SID=<sid>"

# Filter by state
curl "http://localhost:8080/api/v2/torrents/info?
    filter=downloading" \
    --cookie "SID=<sid>"

# Get sync data
curl "http://localhost:8080/api/v2/sync/maindata?rid=0" \
    --cookie "SID=<sid>"

# Get torrent properties
curl "http://localhost:8080/api/v2/torrents/properties?
    hash=<hash>" \
    --cookie "SID=<sid>"

# ===== MANAGEMENT =====
# Pause
curl -X POST http://localhost:8080/api/v2/torrents/stop \
    --cookie "SID=<sid>" -d 'hashes=all'

# Resume
curl -X POST http://localhost:8080/api/v2/torrents/start \
    --cookie "SID=<sid>" -d 'hashes=<hash>'

# Delete (keep files)
curl -X POST http://localhost:8080/api/v2/torrents/delete \
    --cookie "SID=<sid>" -d 'hashes=<hash>' -d 'deleteFiles=false'

# Set category
curl -X POST http://localhost:8080/api/v2/torrents/setCategory \
    --cookie "SID=<sid>" -d 'hashes=<hash>' -d 'category=Movies'

# Add tags
curl -X POST http://localhost:8080/api/v2/torrents/addTags \
    --cookie "SID=<sid>" -d 'hashes=<hash>' -d 'tags=hd,2024'

# ===== CATEGORIES & TAGS =====
# Create category
curl -X POST http://localhost:8080/api/v2/torrents/createCategory
    \
    --cookie "SID=<sid>" -d 'category=Movies' -d 'savePath=/
    downloads/Movies'

# Create tags

```

```

curl -X POST http://localhost:8080/api/v2/torrents/createTags \
  --cookie "SID=<sid>" -d 'tags=movies,tv,hd'

# List tags
curl http://localhost:8080/api/v2/torrents/tags --cookie
  "SID=<sid>"

# ===== APPLICATION =====
# Get version
curl http://localhost:8080/api/v2/app/version --cookie "SID=<sid>"
curl http://localhost:8080/api/v2/app/webapiVersion --cookie
  "SID=<sid>"

# Get/set preferences
curl http://localhost:8080/api/v2/app/preferences --cookie
  "SID=<sid>"
curl -X POST http://localhost:8080/api/v2/app/setPreferences \
  --cookie "SID=<sid>" -d 'json={"dl_limit":10240}'

```

19. Browser Extension Integration Notes

19.1 Cross-Origin Considerations

Browser extensions making requests to qBittorrent must handle CORS:

```

// In manifest v3, use host permissions
// manifest.json
{
  "host_permissions": [
    "http://*/",
    "https://*/"
  ],
  "permissions": [
    "storage",
    "activeTab",
    "contextMenus"
  ]
}

```

19.2 Storage for qBittorrent Settings

```

// Save settings
chrome.storage.sync.set({
  qbBaseUrl: 'http://localhost:8080',
  qbUsername: 'admin',
  qbPassword: 'adminadmin', // Consider using
                           chrome.storage.local for better security
})

```

```

    defaultCategory: 'General',
    defaultSavePath: '/downloads',
    autoStart: true,
    sequentialDownload: false,
  });

  // Retrieve settings
  const settings = await chrome.storage.sync.get([
    'qbBaseUrl', 'qbUsername', 'qbPassword', 'defaultCategory'
  ]);

```

19.3 Context Menu for Magnet/Torrent Links

```

// background.js - Create context menu
chrome.runtime.onInstalled.addListener(() => {
  chrome.contextMenus.create({
    id: 'add-to-qbittorrent',
    title: 'Add to qBittorrent',
    contexts: ['link'],
    targetUrlPatterns: [
      'magnet:*',
      '*://*.torrent',
      '*://*/download/*.torrent',
    ],
  });
});

chrome.contextMenus.onClicked.addListener((info, tab) => {
  if (info.menuItemId === 'add-to-qbittorrent') {
    const url = info.linkUrl;
    if (url.startsWith('magnet:')) {
      addMagnet(url);
    } else if (url.endsWith('.torrent')) {
      downloadAndAddTorrent(url);
    }
  }
});

```

19.4 Security Considerations

1. **HTTPS:** Always use HTTPS for remote qBittorrent instances
 2. **Credentials:** Store credentials in `chrome.storage.local` (not `sync`) and consider encrypting
 3. **CSRF:** The `Referer` header must match the qBittorrent host. The extension's background script can set this correctly.
 4. **Authentication failures:** Implement exponential backoff for failed login attempts to avoid IP bans
-

20. References

Primary Documentation

Source	URL
WebUI API (qBittorrent 4.1) Wiki	https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-(qBittorrent-4.1)
WebUI API (qBittorrent 5.0) Wiki	https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-(qBittorrent-5.0)
API Key Authentication Wiki	https://github.com/qbittorrent/qBittorrent/wiki/API-Key-Authentication-(%E2%89%A5v5.2.0)
qbittorrent-api Python Library	https://qbittorrent-api.readthedocs.io/
qbittorrent-api PyPI	https://pypi.org/project/qbittorrent-api/

Key Citations from Research

[⁶]: GitHub Wiki - “WebUI API (qBittorrent 4.1)” - Complete API documentation for qBittorrent v4.1-v4.6.x. [https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-\(qBittorrent-4.1\)](https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-(qBittorrent-4.1))

[⁴⁷]: GitHub Issue #21106 - “Incorrectly tripping CSRF protection in WebUI when using restrictive Referrer-Policy” - Documents CSRF protection behavior and Origin/Referer header requirements. <https://github.com/qbittorrent/qBittorrent/issues/21106>

[⁴⁹]: GitHub Issue #10999 - “api/v2/sync/maindata is much more slower than api/v2/torrents/info” - Performance note about sync endpoint with large torrent counts. <https://github.com/qbittorrent/qBittorrent/issues/10999>

[¹¹⁹]: GitHub Wiki - “API Key Authentication (>=v5.2.0)” - Documents new Bearer token authentication. [https://github.com/qbittorrent/qBittorrent/wiki/API-Key-Authentication-\(%E2%89%A5v5.2.0\)](https://github.com/qbittorrent/qBittorrent/wiki/API-Key-Authentication-(%E2%89%A5v5.2.0))

[¹²¹]: GitHub Wiki - “WebUI API (qBittorrent 5.0)” - API documentation for v5.0+ with state name changes. [https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-\(qBittorrent-5.0\)](https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-(qBittorrent-5.0))

[¹³⁷]: qbittorrent-api ReadTheDocs - “Definitions” - TorrentState enum with all state values and v5.0 rename notes. <https://qbittorrent-api.readthedocs.io/en/latest/apidoc/definitions.html>

[³⁹]: qbittorrent-api ReadTheDocs - “Search” - Search API implementation details. <https://qbittorrent-api.readthedocs.io/en/latest/apidoc/search.html>

[⁵⁹]: qbittorrent-api ReadTheDocs - “RSS” - RSS API implementation details. <https://qbittorrent-api.readthedocs.io/en/latest/apidoc/rss.html>

[⁶²]: qbittorrent-api PDF documentation (v2025.5.0) - Complete API reference. https://qbittorrent-api.readthedocs.io/_/downloads/en/v2025.5.0/pdf/

[¹⁶⁴]: GitHub Wiki - “Get torrent list” section - Full torrent info response fields. [https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-\(qBittorrent-4.1\)#get-torrent-list](https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-(qBittorrent-4.1)#get-torrent-list)

[¹⁶⁶]: GitHub Wiki (v5.0) - “Get torrent list” - Updated state values for v5.0+. [https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-\(qBittorrent-5.0\)#get-torrent-list](https://github.com/qbittorrent/qBittorrent/wiki/WebUI-API-(qBittorrent-5.0)#get-torrent-list)

Document Version: 1.0 **Last Updated:** 2025-07-28 **Scope:** Complete qBittorrent WebUI API v2 reference for browser extension integration **Compatibility:** qBittorrent v4.1.0+ through v5.2.1+